

January 2026 CSE 314: Operating System Sessional

Assignment 2: xv6 - System Call

Department of Computer Science and Engineering
Bangladesh University of Engineering and Technology

July 2026

Overview

Suppose our favorite OS xv6 is under virus attack. The virus is calling various system calls on its own. That's why we need to track the system calls with the relevant history of each system call. In this assignment, you will implement new system calls in xv6, which will help you understand how they work and expose you to some of the internals of the xv6 kernel. There are two main tasks:

1. Implement a system call to trace individual system calls for a process
2. Implement a system call to monitor and collect aggregated system call statistics

Change Log

Description	Updated By	Timestamp
Assignment declared	NTD	23 July, 2026, 11:00 AM

1 Task 1: Trace

1.1 Task Overview

Implement a new system call `trace` that will control your program tracing. It should take one argument, an integer `syscall_number` which denotes the system call number to trace for a user program. For example, to trace the `fork` system call, a user program calls `trace(SYS_fork)`, where `SYS_fork` is the syscall number of `fork` from `kernel/syscall.h`.

You have to modify the xv6 kernel to print out a line when each system call is about to return for a process, if the current system call's number is the same as the argument passed in the `trace` system call. The line should contain the following:

- **process id:** a number representing the id of the running process
- **the name of the system call:** a string
- **the arguments of the system call:** a tuple that shows output corresponding to each argument's datatype
- **the return value of the system call:** a number

The `trace` system call should enable tracing for the process that calls it but should **not** affect other processes.

1.2 Requirements

1.2.1 User Program

The following `trace.c` user program runs another program with tracing enabled. For example, when you run `trace 7 echo hello`, the `trace` user program will first enable tracing for syscall number 7 (i.e., `exec`) and run the `echo hello` program. Make necessary changes so that you can use this `trace.c` user program in the shell.

```

1 #include "kernel/param.h"
2 #include "kernel/types.h"
3 #include "kernel/stat.h"
4 #include "user/user.h"
5
6 int
7 main(int argc, char *argv[])
8 {
9     int i;
10    char *nargv[MAXARG];
11
12    if(argc < 3 || (argv[1][0] < '0' || argv[1][0] > '9')){
13        fprintf(2, "Usage: %s sys_call_num command\n", argv[0]);
14        exit(1);
15    }
16
17    if (trace(atoi(argv[1])) < 0) {
18        fprintf(2, "%s: trace failed\n", argv[0]);
19        exit(1);
20    }
21
22    for(i = 2; i < argc && i < MAXARG; i++){
23        nargv[i-2] = argv[i];

```

```

24 }
25 exec(nargv[0], nargv);
26 exit(0);
27 }

```

You might expect output like this:

```

xv6 kernel is booting

hart 1 starting
hart 2 starting
init: starting sh
$ trace 15 grep hello README
pid: 3, syscall: open, args: (README, 0), return: 3
$ grep hello README
$ trace 3 grep hello README
$ trace 5 grep hello README
pid: 4, syscall: read, args: (3, 0x0000000000001010, 1023), return: 1023
pid: 4, syscall: read, args: (3, 0x000000000000103a, 981), return: 981
pid: 4, syscall: read, args: (3, 0x0000000000001023, 1004), return: 350
pid: 4, syscall: read, args: (3, 0x0000000000001010, 1023), return: 0
$ trace 21 grep hello README
pid: 5, syscall: close, args: (3), return: 0
$ trace 7 echo hello
pid: 6, syscall: exec, args: (echo, 0x00000000000003e60), return: 2
hello
$

```

1.2.2 Explanation

- **trace 15 grep hello README:** It means you want to trace system call number 15 (i.e., **open**) for the program **grep hello README**. You can see 1 line of output, so it called the **open** system call 1 time. The arguments to **open** are: a string representing the path to the file (here, **README**) and an integer representing some flags (here, 0). The arguments are printed in their expected format.
- **grep hello README:** It has no tracing output because it did not set the tracing through the trace user program.
- **trace 3 grep hello README:** It sets trace for system call number 3 (i.e., **wait**) which does not print anything as **wait** is never called for **grep hello README**.
- **trace 5 grep hello README:** It sets trace for system call number 5 (i.e., **read**) which occurs 4 times for **grep hello README**. **read** takes a file descriptor (integer), a destination for reading data (pointer), and number of bytes to read (integer) as arguments. The arguments are printed in their expected format.
- **trace 21 grep hello README:** It traces **close** system call which is called once here (as the file **README** will be closed once after reading it). **close** takes a file descriptor (integer) as argument.

- `trace 7 echo hello`: It traces `exec` system call which is called once here (as the `echo` command will be executed once). `exec` takes the name of the program (string) and command line arguments to that program (pointer). The arguments are printed in their expected format.

Hints

- You might need to add an extra field in the `proc` structure (see `kernel/proc.h`) to remember which system call the process wants to trace.
- The `syscall()` function in `kernel/syscall.c` is the point where the kernel decides which system call handler to invoke for a specific system call number, calls that handler and returns the return value. So it is a perfect place to modify to print the trace output. To print the syscall name, you might need to add an array of syscall names in that file to index into.
- All the arguments to system calls go through one of: `argint`, `argaddr`, `argstr`. So, printing the arguments in their corresponding type should be easy.

2 Task 2: System Call History

2.1 Task Overview

Implement a new system call `history` that will return the aggregated history of system calls (how many times each was called and the system time it consumed). It should take one argument, an integer `syscall_number` (and possibly another argument, described in section 2.2.1) which denotes the system call number to trace. For example, to get history about the `fork` system call, a user program calls `history(SYS_fork)`, where `SYS_fork` is the syscall number from `kernel/syscall.h`.

2.2 Requirements

2.2.1 System Call Implementation

After the return from your system call, you have a pointer (defined in user program) to a `struct` object. This `struct` object should contain:

1. The name of the system call
2. The number of times this system call was made
3. The total time consumed from boot-up by this system call

A sample structure could be:

```
1 struct syscall_stat {
2     char syscall_name[16];
3     int count;
4     int accum_time;
5 };
```

And the corresponding system call signature - `int history(int, struct syscall_stat*)`;

2.2.2 User Program

You must implement a `history.c` user program that calls this system call and displays the results. For example, when you run `history 5`, the history user program will fetch the necessary info from the kernel and then print in the console from user mode.

A sample output might look like this (some values may vary):

```
xv6 kernel is booting

hart 2 starting
hart 1 starting
init: starting sh
$ history 12
12: syscall: sbrk, #: 1, time: 0
$ history 5
5: syscall: read, #: 21, time: 58
$ history 5
5: syscall: read, #: 31, time: 91
$ history 22
22: syscall: history, #: 3, time: 0
```

```
$ history
1: syscall: fork, #: 6, time: 0
2: syscall: exit, #: 0, time: 0
3: syscall: wait, #: 4, time: 1
4: syscall: pipe, #: 0, time: 0
5: syscall: read, #: 50, time: 278
6: syscall: kill, #: 0, time: 0
7: syscall: exec, #: 7, time: 0
8: syscall: fstat, #: 0, time: 0
9: syscall: chdir, #: 0, time: 0
10: syscall: dup, #: 2, time: 0
11: syscall: getpid, #: 0, time: 0
12: syscall: sbrk, #: 5, time: 0
13: syscall: sleep, #: 0, time: 0
14: syscall: uptime, #: 0, time: 0
15: syscall: open, #: 3, time: 1
16: syscall: write, #: 670, time: 2
17: syscall: mknod, #: 1, time: 0
18: syscall: unlink, #: 0, time: 0
19: syscall: link, #: 0, time: 0
20: syscall: mkdir, #: 0, time: 0
21: syscall: close, #: 1, time: 0
22: syscall: history, #: 25, time: 0
```

2.2.3 Explanation

- `history 12` means return history for system call 12. `sbrk` is the system call for index 12 (refer to `syscall.h`). It has been called only once. Moreover, the time (xv6 ticks) consumed by it is 0.
- Notice the difference between the outputs of two `history 5` calls as more system calls occur between them.
- `history 22` confirms our implementation is correct. Here we have used number 22 for the history system call.
- Finally, `history` without any arguments should print history for all system calls.
- You might see `trace` in the output when you type `history` only and press enter.

All information must be printed from **user mode**. Printing anything in kernel mode is not allowed.

Hints

- Please refer to the system call `fstat` to get an idea of how to return a structure object pointer.
- You need to modify `syscall.c` to count the occurrences of each system call and measure their execution time.
- Introduce the appropriate data structures to globally track system call statistics. Initialization of these structures may be necessary; `kernel/main.c` is a suitable location for such initialization (refer to the `procinit()` function for guidance about locks).

- You might need to add functions in the kernel to maintain the counters and timers for each system call.

2.2.4 Locking

Xv6 is a multiprocessor system. There is a variable `CPUS` in `Makefile`. If two processes running the same system call on different CPUs increment the counter for this system call at the exact same time, this may lead to one update not being recorded.

An effective approach to addressing this issue involves the use of locks. Refer to the implementation of `tickslock` in `xv6` as a reference for employing locking mechanisms. To ensure the operating system maintains high efficiency, fine-grained locking techniques should be utilized.

3 General Guidelines

- Don't forget to acquire and release locks when needed. Look out for the `proc` struct in `kernel/proc.h` and `kmem` struct in `kernel/kalloc.c`. You should look at how other existing functions use the fields of those structs to get an idea.
- Remember `xv6` is multi-core, so proper synchronization is essential.
- Make incremental changes and test them thoroughly to avoid debugging complex issues.
- Read the `xv6` book to better understand the internals and the existing implementation.

4 Submission Guidelines

For this lab assignment, you must start with a fresh copy of the `xv6` repository available at <https://github.com/shuaibw/xv6-riscv>. Clone the repository using the following command:

```
1 git clone https://github.com/shuaibw/xv6-riscv --depth=1
```

Using a fresh copy of this repository is important because, during the lab evaluation, you may be asked to modify and regenerate your patch file.

After cloning the repository, make all the necessary modifications and create any additional files required for the assignment. Do not commit your changes. Once you have completed and tested your implementation, generate a patch file containing only your changes using the following commands from inside the `xv6` repository:

```
1 git add --all
2 git diff HEAD > studentID.patch
```

Replace `studentID` with your own seven-digit student ID. For example, if your student ID is 2205192, the patch file should be named `2205192.patch`.

Submit only the patch file. Do not submit the entire `xv6` repository, and do not compress or zip the patch file.

During the lab evaluation, we will clone a fresh copy of the same `xv6` repository and apply your submitted patch using the following command:

```
1 git apply studentID.patch
```

Before submitting, verify that your patch works correctly by applying it to another fresh copy of the repository using the same procedure that will be followed during the lab evaluation.

Please DO NOT COPY solutions from anywhere (your friends, seniors, internet, etc.). Any form of plagiarism (irrespective of source or destination), will result in getting -100% marks in this assignment. You have to protect your code.

Submission Deadline

Friday, July 31, 2026, 11:45 PM

Mark Distribution

Task	Sub-task	Marks
Trace	Properly tracing system calls	15
	Tracing only for the calling process	10
	Printing system call name	5
	Printing with system call arguments	10
History	Designing history.c	10
	Counting system calls	15
	Calculating system call time	15
	Using appropriate locking	15
Proper submission		5
Total		100